14. Conditional Rendering in React

▼ What is Conditional Rendering in React?

- Conditional Rendering means showing different UI based on a condition.
- For example, you may want to show a loading spinner while fetching data from an API, or you may want to show a login button for unauthenticated users and a logout button for authenticated users.

▼ Real-Life Examples

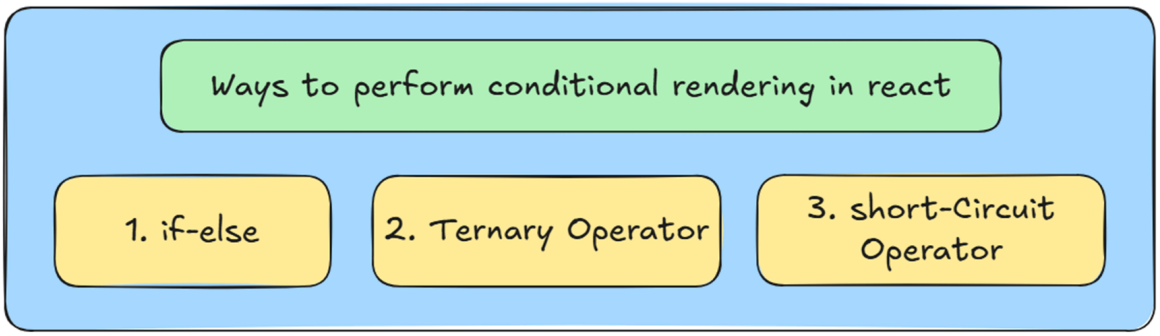
- Show **Login** button if user is not logged in
- Show **Logout** button if user is logged in
- Show **Loading...** while fetching data
- Show **Error message** if something goes wrong

▼ Why Do We Need Conditional Rendering?

- Dynamically control **what the user sees** (based on state, props, or logic).
- Improve **UX** (like showing a loader, error, or content).
- Keep code **clean** by separating UI logic.

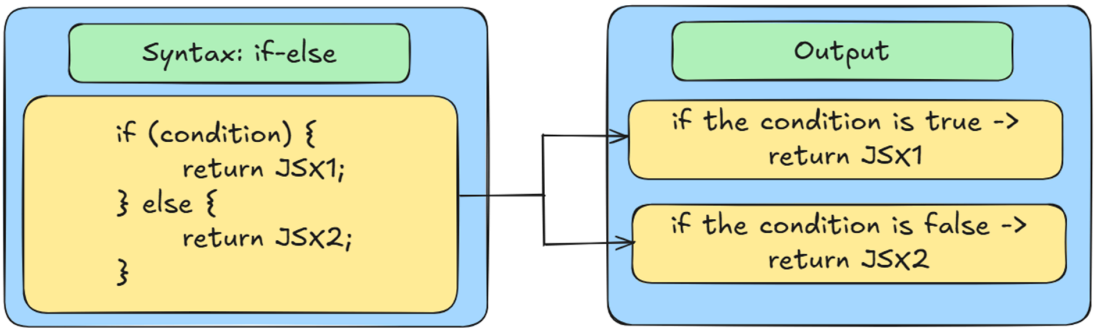
▼ Ways to Perform Conditional Rendering in React

- We can perform conditional rendering in 3 ways in React:



▼ 1. By using `if-else` statement:

- One way of using conditional rendering is by using if-else statements.
- `Important`: If-else **cannot be** used inside JSX.
- It works outside the **return()** statement of the component.
- **Syntax:**



- For example, suppose we have a component called `Greeting` that takes a prop called `name` and renders a greeting message based on whether the name is provided or not. We can use an if-else statement to check the name prop and return different elements accordingly.
- **Code: App.jsx**

```
import React from "react";
import Greetings from "./Conditional Rendering/Greetings";

const App = () => {
  return (
    <div>
      <Greetings name="John" />
```

```
    </div>
  );
};

export default App;
```

- **Code: Greetings.jsx**

```
import React from "react";

const Greetings = (props) => {
  let { name } = props;

  if (name) {
    return <h1>Hello, {name}!</h1>;
  } else {
    return <h1>Hello, Stranger!</h1>;
  }
};

export default Greetings;
```

- **Output:**

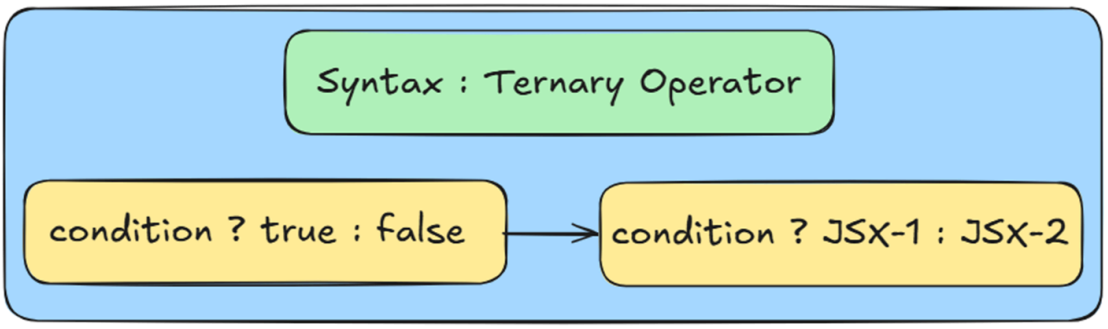
Hello, John!

- In this example, we use an if-else statement to check if the name prop is true (not null, undefined, empty string, etc.). If it is true, we return an `<h1>` element with the name. If it is false, we return an `<h1>` element with the word “stranger”.

▼ 2. By using `Ternary Operator`:

- The ternary operator is used as a **shorter syntax** for the if-else statement.
- It works inside the **JSX** (`{}`).

- This operator can be used to render one of two elements based on a condition.
- **Syntax:**



- For Example → Suppose we have a component called `User` that has a state called `isLoggedIn` and renders a button based on the user's authentication status. We can use the ternary operator to render either a login button or a logout button.
- **Code:**

```
import React, { useState } from "react";

const User = () => {
  const [isLoggedIn, setIsLoggedIn] = useState(true);
  // true → Logout
  // false → Login
  return (
    <div>
      <h1>Conditional Rendering by using - Ternary Operator</h1>
      {isLoggedIn ? <button>Logout</button> : <button>Login</button
>}
    </div>
  );
};

export default User;
```

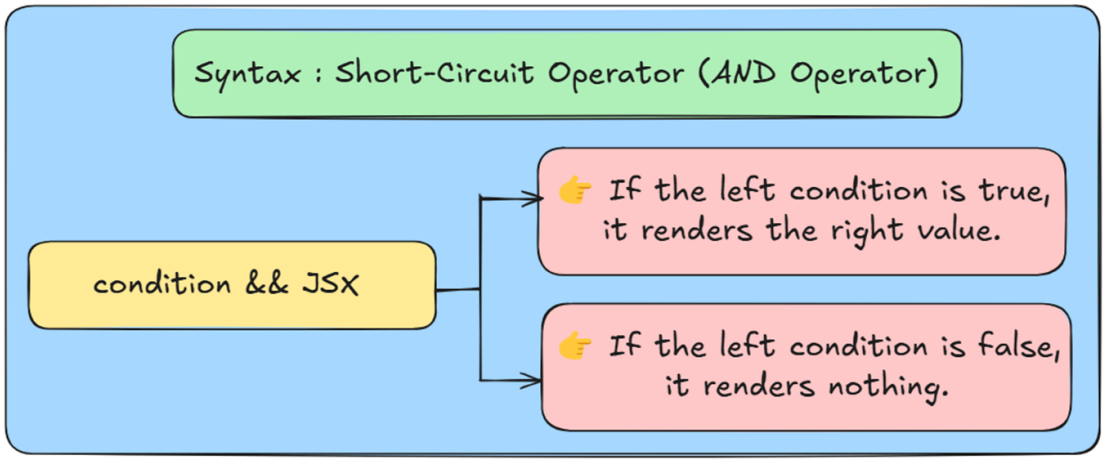
- **Output:**

Conditional Rendering by using - Ternary Operator

Logout

▼ 3. By using Short-Circuit (`&&`) Operator:

- Another way of using conditional rendering is by using the `&&` operator.
- Short Circuit (`&&`) (`AND Operator`).
- The short-circuit operator works from **left to right**.
- It works inside the **JSX** (`{}`).
- This operator can be used to render an element only when a condition is `true` .
- Syntax:



- For Example → Suppose we have a component called `TodoList` has state called items and render a list of todo items.
- We can use the `&&` operator to render a message when the list is empty.
- **Code:**

```
import React, { useState } from "react";

const TodoList = () => {
  let [items, setItems] = useState([
```

```
    "Buy groceries",
    "Complete React assignment",
    "Call the electrician",
    "Workout for 30 minutes",
    "Read 10 pages of a book",
  ]);
  return (
    <div>
      <h1>Todo List</h1>
      <ul>
        {items.length > 0 &&
          items.map((item, index) => {
            return <li key={index}>{item}</li>;
          })}
      </ul>
    </div>
  );
};

export default TodoList;
```

- **Output:**

Todo List

- Buy groceries
- Complete React assignment
- Call the electrician
- Workout for 30 minutes
- Read 10 pages of a book